TECHNICAL ARCHITECTURE, AUDIT & WORKFLOW SPECIFICATION

RAG-Powered AI Teaching Assistant

Comprehensive architectural and workflow specification for an on-device, zero-cost Retrieval-Augmented Generation (RAG) system. Engineered with LangChain (v0.3), ChromaDB (v0.5), FastAPI, Streamlit, and local quantized LLM inference via Ollama (Meta Llama 3.2 3B and Nomic Embed Text). Includes verified code audits, error matrix, technical debt evaluation, limitations, and a prioritized engineering roadmap.

Project Name:	AI Teaching Assistant (Package: ai-teaching-assistant)
Author / Role:	Senior Software Architect & Technical Lead
Current Architecture Status:	**Local Prototype / Functional Proof-of-Concept (PoC)** (Verified for single-user local educational use; not enterprise production-ready)
Codebase Location:	C:\Users\naren\IT VEDANT PROJECT\ai-teaching-assistant
Host Environment:	Microsoft Windows 11 / Python 3.13 / Ollama Engine (:11434)
Audit Date:	October 15, 2026

REALISTIC STATUS & EXECUTIVE AUDIT SUMMARY

Architectural Realignment: While earlier documentation drafts described this system as 'Production Ready', an objective engineering audit confirms that this codebase is a **high-performing Local Development Prototype and Proof-of-Concept (PoC)**. All primary functional workflows (document ingestion, chunking, Ollama vectorization, ChromaDB storage, MMR retrieval, and citation Q&A;) are completely operational and verified locally. However, enterprise production characteristics (such as multi-user authentication, tenant isolation, asynchronous background queues for large files, and automated unit test suites) are not yet implemented.

PART 1: RIGOROUS CODE VERIFICATION FINDINGS

Audit Item	Code Inspection & Empirical Result	Architectural Consequence
(a) Chroma Distance Metric & Location	In ingestion_service.py:20, get_vectorstore() creates Chroma without collection_metadata. Direct inspection of chroma.sqlite3 confirms Chroma defaults to **Squared L2 Norm (space='l2')**, NOT cosine distance.	Vector retrieval computes Euclidean distance. Because nomic-embed-text generates normalized vectors, L2 distance is monotonically related to cosine distance, but explicit configuration is standard.
(b) Page Metadata for DOCX & UI Indexing	In ingestion_service.py:load_document(), Docx2txtLoader returns {'source': filename} with **NO page metadata**. In chat.py:59, page is None. In chat_interface.py:171, page_info is omitted. For PDFs, PyMuPDFLoader produces 0-indexed pages; chat.py converts page + 1. **PDFs display 1-indexed pages; DOCX displays no page number**.	Consistent user presentation: PDF sources show '■ file.pdf • Page 4', while DOCX sources display solely '■ notes.docx'.
(c) Duplicate Upload Handling	In documents.py, raw files are overwritten via open(..., 'wb'). In ingestion_service.py:91, vectorstore.add_documents() generates **new random UUIDs** without pre-deleting existing chunks for that filename.	**Uploading the same file twice duplicates chunks in ChromaDB**. Total chunk count doubles, wasting storage and causing MMR to evaluate identical redundant chunks.
(d) Compatibility with .doc Files	While .doc is listed in ALLOWED_EXTENSIONS, Docx2txtLoader delegates to docx2txt which unzips OpenXML packages. Legacy binary Word 97-2003 OLE files fail with zipfile.BadZipFile: File is not a zip file.	**Legacy binary .doc files DO NOT work**. They fail ingestion with an unhandled exception converted to HTTP 500. Only modern .docx works.

PART 2: ERROR & EDGE CASE MATRIX

Scenario / Edge Case	Trigger / Root Cause	Backend Behavior (Code Execution)	Frontend UI Presentation
Ollama Offline	Daemon stopped (port 11434 unreachable).	/health returns status='degraded'. In /upload, embedding fails (HTTP 500). In /chat/, catches ConnectionError (HTTP 503).	Sidebar renders ■ Ollama badge. Chat displays error: '■ Ollama is not running. Run: ollama serve'.
Model Not Pulled	Model tag missing in Ollama.	Ollama returns 404. In documents.py or chat.py, uncaught exception becomes HTTP 500 with message.	UI displays: '■ Upload failed: model nomic-embed-text not found' or '■ Error: Query failed: model not found'.
Scanned Image PDF	PDF contains images of text without OCR layer.	load_document() evaluates total_chars < 50, unlinks file, raises HTTP 422 Unprocessable Entity.	UI shows: '■ Upload failed: The file appears to be empty or is a scanned image PDF. Please use text-based PDF.'
Empty Database	User queries before uploading documents.	In chat.py, get_collection_count() == 0 raises HTTP 400 Bad Request.	UI renders warning: '■■ No documents found in the knowledge base. Please upload PDF or DOCX files first.'
Oversized File (>50MB)	Uploaded file exceeds 50MB.	In documents.py, byte check aborts prior to disk save with HTTP 413 Content Too Large.	UI displays: '■ Upload failed: File too large (XMB). Maximum allowed: 50MB'.
Duplicate Upload	Same file uploaded a second time.	Disk file overwritten. Chroma add_documents() creates new UUIDs, doubling chunks.	UI displays success. Document list still shows 1 file, but sidebar chunk count increments.
Legacy .doc Upload	Word 97-2003 binary file uploaded.	docx2txt raises zipfile.BadZipFile. Caught by generic exception, returns HTTP 500.	UI displays: '■ Upload failed: Ingestion failed: File is not a zip file. Is Ollama running?'
Inference Timeout	Slow local CPU generation exceeds 180s.	Streamlit httpx.post() hits 180s timeout; ChatOllama has 120s timeout.	UI displays: '■■ The response timed out. The model might be too slow. Try llama3.2.'

PART 3: FULL ARCHITECTURE & SYSTEM SCHEMATICS

3.1 Component & Module Breakdown

Module Path	Key Responsibilities	Inputs / Outputs	Dependencies
backend.core.config	Loads .env via Pydantic BaseSettings. Auto-creates data directories.	In: .env file Out: Settings singleton	pydantic_settings, os
backend.models.schemas	Defines request/response data contracts and validation boundaries.	In: Raw JSON/Forms Out: Validated DTO objects	pydantic
backend.services.ingestion_service	Parses PDF/DOCX, splits text (800/150), calls Ollama embeddings, stores in Chroma.	In: File path, filename Out: Created chunk count	PyMuPDF, Docx2txt, RecursiveSplitter, Chroma, OllamaEmbeddings
backend.services.llm_service	MMR retriever (k=4, fetch_k=15), injects context into TEACHING_PROMPT, invokes LLM.	In: Question string, model Out: (Answer string, Source docs)	ChatOllama, Chroma, LCEL runnables, StrOutputParser
backend.api.v1.endpoints.documents	Endpoints: POST /upload (max 50MB), GET / (list files), DELETE /clear (reset DB).	In: Multipart file, HTTP requests Out: Upload/List/Clear JSON	FastAPI, ingestion_service
backend.api.v1.endpoints.chat	Validates non-empty DB, runs RAG query, deduplicates citations by (source, page).	In: ChatRequest JSON Out: ChatResponse JSON	FastAPI, llm_service, ingestion_service
frontend.components.sidebar	Renders model dropdown, file dropzone, doc list, health status, and danger zone.	In: User UI clicks Out: HTTP calls to FastAPI	Streamlit, HTTPX
frontend.components.chat_interface	Renders chat history, user input, thinking spinners, and expandable source citations.	In: Chat text input Out: Rendered markdown & state	Streamlit, HTTPX

3.2 Complete REST API Documentation

Method & Endpoint	Request Format	Success Response	Error Codes	Handler Location
GET /health	None	HealthResponse (status, ollama_ok, doc_count)	None ('degraded')	backend/main.py
POST /api/v1/documents/upload	multipart/form-data: file (PDF/DOCX)	UploadResponse (chunks_created, success=true)	400 (ext), 413 (size), 422 (scanned), 500	backend/api/v1/endpoints/documents.py
GET /api/v1/documents/	None	DocumentsListResponse (array of DocumentInfo)	500	backend/api/v1/endpoints/documents.py
DELETE /api/v1/documents/clear	None	ClearResponse (files_deleted, success=true)	500	backend/api/v1/endpoints/documents.py
POST /api/v1/chat/	JSON: question, model, top_k	ChatResponse (answer, sources array, model_used)	400 (empty DB), 503 (Ollama), 500	backend/api/v1/endpoints/chat.py

3.3 Quick Reference Mapping: Feature → Files → Endpoints / DB

Feature	Key Files Involved	Primary Functions / Classes	Bound Endpoint / DB Table
Document Upload	sidebar.py, documents.py, ingestion_service.py	_handle_upload(), upload_document(), ingest_document()	POST /api/v1/documents/upload raw_documents/
Text Chunking	ingestion_service.py	RecursiveCharacterTextSplitter.split_documents()	In-memory transform
Vector Indexing	ingestion_service.py	get_vectorstore(), vectorstore.add_documents()	Chroma collection: teaching_materials (HNSW)
RAG Q&A;	chat_interface.py, chat.py, llm_service.py	handle_user_input(), chat(), query_rag()	POST /api/v1/chat/
MMR Retrieval	llm_service.py	get_retriever(), retriever.invoke()	Chroma query (k=4, fetch_k=15)
Citation Generation	chat.py, chat_interface.py	chat() deduplicator, _render_sources()	SourceDocument DTO
Clear Database	sidebar.py, documents.py, ingestion_service.py	clear_all(), clear_all_documents()	DELETE /api/v1/documents/clear
System Health	sidebar.py, main.py	_render_health_status(), health_check()	GET /health

PART 4: LIMITATIONS, TECHNICAL DEBT & ROADMAP

4.1 Architectural Limitations & Technical Debt

- **Single-Tenant Architecture:** Zero authentication, authorization, or user isolation. All users access and mutate the same vector collection.
- **Synchronous Ingestion Blocking:** Document loading and vector generation execute synchronously inside the FastAPI threadpool. Ingesting large 200-page textbooks blocks worker resources.
- **Duplicate Upload Inefficiency:** Uploading an identical document creates duplicate chunks in ChromaDB without idempotency checks.
- **Non-Streaming Chat Latency:** LLM responses are returned as a single block after full completion. Users experience 10-30 seconds of perceived latency on CPU hardware.
- **Memory-Bound Session State:** Chat history resides entirely in Streamlit RAM (st.session_state). Refreshing the browser tab permanently clears conversation history.
- **Inaccurate .doc Support:** Code accepts .doc in allowlists but underlying Docx2txtLoader crashes on legacy binary Word files.
- **Unverified Hardware Scenarios:** Performance on external dedicated GPUs, encrypted PDFs, or multi-user concurrent loads has *Not been verified*.

4.2 Prioritized Improvement Roadmap

Priority	Milestone / Initiative	Technical Implementation Details
P0 (Immediate)	Ingestion Idempotency & Duplicate Purge	Invoke delete_document_chunks(filename) prior to add_documents(). Prevents duplicate chunks in ChromaDB upon re-uploading.
P0 (Immediate)	Fix .doc Extension Handling	Remove .doc from allowlists or integrate antiword/catdoc. Display clear error that only OpenXML .docx is supported.
P1 (Near-Term)	Streaming Token Responses	Implement Server-Sent Events (SSE) or WebSockets in FastAPI using LangChain chain.astream() to stream tokens directly into Streamlit.
P1 (Near-Term)	Persistent Chat History	Add SQLite or DuckDB conversational memory table to preserve user chat threads across browser reloads.
P2 (Production)	Background Task Worker (Celery / RQ)	Offload heavy document ingestion to background task workers with Redis, returning a task ID and polling status bar.
P2 (Production)	Multi-Tenancy & JWT Auth	Implement OAuth2/JWT authentication and isolate vector collections per user ID or course ID.

4.3 Technical Glossary

- **RAG (Retrieval-Augmented Generation):** Combining information retrieval with text synthesis to ground LLMs in external knowledge.
- **LCEL (LangChain Expression Language):** Declarative composition syntax for chaining prompts, models, and parsers (e.g. prompt | llm | parser).
- **MMR (Maximal Marginal Relevance):** A retrieval algorithm balancing relevance to the query with diversity among retrieved items.
- **Bi-Encoder:** A transformer model (e.g. nomic-embed-text) encoding queries and documents independently into vector space.
- **HNSW (Hierarchical Navigable Small World):** A multi-layer graph-based indexing algorithm for ultra-fast approximate nearest neighbor search.
- **GGUF / Quantization:** Compression format converting 16-bit model weights to 4-bit (Q4_K_M) for high-speed local inference.